

Chuẩn_hóa_dữ_liệu

June 15, 2026

1 Chuẩn hóa dữ liệu

1.1 Giới thiệu

Dữ liệu gồm nhiều thuộc tính (cột), và mỗi thuộc tính thì lại có các đơn vị và độ lớn nhỏ khác nhau. Điều này tác động tới tính hiệu quả của nhiều thuật toán, ví dụ thời gian thực hiện, quá trình hội tụ, hay thậm chí ảnh hưởng cả tới độ chính xác của thuật toán.

Ví dụ: xét hàm tính khoảng cách giữa hai điểm (x_1, y_1) và (x_2, y_2)

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Giả sử rằng $x \in [0, 1]$ và $y \in [0, 1000]$, khi đó, khoảng cách giữa hai điểm phụ thuộc hoàn toàn vào giá trị của y .

Điều này dẫn tới, nếu như thuật toán có dùng đến hàm tính khoảng cách như trên thì vai trò của x sẽ trở nên không đáng kể.

Vì vậy, dữ liệu cần được đưa về một khoảng giá trị phù hợp.

1.2 Kỹ thuật thường dùng

Tùy thuộc vào tính chất cũng như khoảng giá trị, chúng ta có một số kỹ thuật: - **Chuẩn hóa bằng min-max** - **Chuẩn tắc hóa dữ liệu** (Chuẩn hóa bằng điểm Z). - **Chuẩn hóa bằng phân vị** - Chuẩn hóa theo số chữ số - Chuẩn hóa theo log - ...

1.3 Chuẩn bị

Trong phần này, chúng ta sẽ sử dụng đến bộ dữ liệu iris và thư viện scikit-learn.

```
[1]: import seaborn as sns
from sklearn.preprocessing import (MinMaxScaler, #chuẩn hóa bằng min-max
                                   StandardScaler, #chuẩn hóa bằng điểm Z
                                   RobustScaler)    #chuẩn hóa bằng phân vị

%matplotlib inline
```

```
[2]: # tải dữ liệu
iris = sns.load_dataset('iris')
# lấy ra cột 'sepal_length' để làm việc, đổi tên cột thành 'before'
data = iris[['sepal_length']].rename({'sepal_length': 'before'}, axis = 0
                                     ↪ 'columns')
```

```
data.head()
```

```
[2]: before
0    5.1
1    4.9
2    4.7
3    4.6
4    5.0
```

2 Các kỹ thuật chuẩn hóa

2.1 Chuẩn hóa bằng min-max

Chuẩn hóa bằng min-max (normalization) là kỹ thuật chuyển đổi giá trị về khoảng $[0, 1]$ theo công thức:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Chuẩn hóa bằng min-max thường được dùng khi: - Biết được (gần đúng) giá trị lớn nhất và giá trị nhỏ nhất *trong thực tế*. - Khi không chắc chắn về phân phối của dữ liệu.

```
[4]: #làm theo công thức
df = data.copy()
df = df.assign(
    norm = (df.before - data.before.min()) / (df.before.max() - df.before.min())
)
df.head()
```

```
[4]: before    norm
0     5.1  0.222222
1     4.9  0.166667
2     4.7  0.111111
3     4.6  0.083333
4     5.0  0.194444
```

```
[5]: #hoặc có thể dùng MinMaxScaler
d = MinMaxScaler().fit(data) #tạo mô hình và truyền dữ liệu
d.transform(data)[:5] #chuyển đổi dữ liệu
```

```
[5]: array([[0.22222222],
        [0.16666667],
        [0.11111111],
        [0.08333333],
        [0.19444444]])
```

```
[6]: #hoặc làm cả hai cùng lúc
MinMaxScaler().fit_transform(data)[:5]
```

```
[6]: array([[0.22222222],
          [0.16666667],
          [0.11111111],
          [0.08333333],
          [0.19444444]])
```

2.2 Chuẩn hóa bằng điểm Z

2.2.1 Điểm Z (Z-score)

Cho biến ngẫu nhiên X (một thử nghiệm không thể biết kết quả trước khi thực hiện) có giá trị trung bình μ , độ lệch chuẩn là σ , khi đó, điểm Z của một giá trị x được tính theo công thức

$$Z_x = \frac{x - \mu}{\sigma}$$

Nếu như X có phân phối chuẩn, ta có: - 68.3% dữ liệu có $|Z| \leq 1$. - 95.4% dữ liệu có $|Z| \leq 2$. - 99.7% dữ liệu có $|Z| \leq 3$. - ...

Thông thường, những giá trị có $|Z| > 3$ được xem như là các giá trị ngoại lệ (outlier).

2.2.2 Chuẩn hóa bằng điểm Z

Chuẩn hóa bằng điểm Z (standardization) là việc chuyển đổi dữ liệu sang điểm Z tương ứng của nó (bằng cách dùng trung bình và độ lệch chuẩn của mẫu). Khi đó, dữ liệu sẽ có trung bình là 0 và độ lệch chuẩn là 1.

```
[7]: #làm theo công thức
df = df.assign(
    st = (df.before - df.before.mean()) / df.before.std()
)
df.head()
```

```
[7]:   before    norm    st
0     5.1  0.222222 -0.897674
1     4.9  0.166667 -1.139200
2     4.7  0.111111 -1.380727
3     4.6  0.083333 -1.501490
4     5.0  0.194444 -1.018437
```

```
[8]: # thống kê dữ liệu
df.describe().round(2)
```

```
[8]:   before    norm    st
count  150.00  150.00  150.00
mean     5.84    0.43   -0.00
std     0.83    0.23    1.00
min     4.30    0.00  -1.86
25%     5.10    0.22  -0.90
50%     5.80    0.42  -0.05
```

75%	6.40	0.58	0.67
max	7.90	1.00	2.48

```
[9]: #sử dụng StandardScaler
std = StandardScaler().fit(data)
std.transform(data)[:5]
```

```
[9]: array([[ -0.90068117],
          [-1.14301691],
          [-1.38535265],
          [-1.50652052],
          [-1.02184904]])
```

Lưu ý: giữa hai cách làm có sự khác nhau là do `pandas` và `scikit-learn` sử dụng hai ước lượng khác nhau cho độ lệch chuẩn. `pandas` sử dụng **ước lượng không lệch** (unbiased estimation), trong khi `scikit-learn` sử dụng **ước lượng lệch** (biased estimation).

```
[10]: df = df.assign(
        bst = (df.before - df.before.mean()) / df.before.std(ddof = 0) #dùng ước lượng lệch
    )
df.head()
```

```
[10]:   before    norm      st      bst
0     5.1  0.222222 -0.897674 -0.900681
1     4.9  0.166667 -1.139200 -1.143017
2     4.7  0.111111 -1.380727 -1.385353
3     4.6  0.083333 -1.501490 -1.506521
4     5.0  0.194444 -1.018437 -1.021849
```

```
[ ]: df.describe().round(2)
```

```
[ ]:   before    norm      st      bst
count  150.00  150.00  150.00  150.00
mean     5.84    0.43   -0.00   -0.00
std     0.83    0.23    1.00    1.00
min     4.30    0.00  -1.86   -1.87
25%     5.10    0.22  -0.90   -0.90
50%     5.80    0.42  -0.05   -0.05
75%     6.40    0.58    0.67    0.67
max     7.90    1.00    2.48    2.49
```

Sử dụng chuẩn hóa bằng điểm Z khi dữ liệu có phân phối chuẩn hoặc tựa chuẩn.

2.3 Chuẩn hóa bằng phân vị

2.3.1 Tác động của điểm ngoại lệ (outlier)

Các giá trị ngoại lệ đều có tác động lớn tới việc chuẩn hóa dữ liệu.

Ví dụ: Giả sử có 99 điểm dữ liệu nằm trong khoảng $[0, 10]$ và 1 điểm dữ liệu có giá trị 100. Sau khi chuẩn hóa bằng min-max, 99 điểm dữ liệu kia sẽ có giá trị trong khoảng $[0, 0.1]$. Khiến cho đóng góp trong việc tính khoảng cách của những điểm này giảm xuống.

Các giá trị ngoại lệ cũng có tác động tới giá trị trung bình và độ lệch chuẩn của dữ liệu.

```
[11]: import numpy as np

#tạo 100 điểm dữ liệu theo phân phối chuẩn
#trung bình là 10, độ lệch chuẩn là 20
d = np.random.normal(10, 20, 100)

#thêm 5 điểm ngoại lệ
#những điểm này sẽ nằm từ 140 -> 150
odd = np.concatenate([d, np.random.uniform(140, 150, 5)])
```

```
[12]: print('Trung bình cũ: ', round(np.mean(d), 2))
      print('Độ lệch chuẩn cũ: ', round(np.std(d, axis = 0), 2))
```

Trung bình cũ: 7.22
Độ lệch chuẩn cũ: 21.8

```
[13]: print('Trung bình mới: ', round(np.mean(odd), 2))
      print('Độ lệch chuẩn mới: ', round(np.std(odd, axis = 0), 2))
```

Trung bình mới: 13.78
Độ lệch chuẩn mới: 36.25

Có thể thấy rõ ràng ảnh hưởng của giá trị ngoại lệ dù những giá trị này chỉ chiếm một phần nhỏ trong dữ liệu (trong ví dụ này là $< 5\%$).

Tuy nhiên, các giá trị phân vị lại không bị ảnh hưởng đáng kể.

```
[14]: print('Tứ phân vị cũ: ', np.round(np.quantile(d, [0.25, 0.5, 0.75]), 2).
      ↪tolist())
      print('Tứ phân vị mới: ', np.round(np.quantile(odd, [0.25, 0.5, 0.75]), 2).
      ↪tolist())
```

Tứ phân vị cũ: [-7.14, 8.39, 20.75]
Tứ phân vị mới: [-6.57, 9.82, 21.6]

Một cách xác định các điểm ngoại lệ này là sử dụng điểm Z ($|Z| > 3$).

```
[15]: odd_z = (odd - np.mean(odd)) / np.std(odd)
# lấy ra điểm ngoại lệ
print(odd[np.abs(odd_z) > 3].tolist())
# tính tỷ lệ điểm ngoại lệ
print(round((np.abs(odd_z) > 3).sum() / odd.shape[0], 4))
```

[142.8886853859285, 146.1744712600084, 149.64549715974462, 140.14257215042983,
146.07992163005773]
0.0476

Có hai cách thường dùng để giải quyết các điểm ngoại lệ này: - Bỏ các điểm ngoại lệ (không khuyến khích). - Sử dụng chuẩn hóa bằng phân vị.

2.3.2 Chuẩn hóa bằng phân vị

Chuẩn hóa bằng phân vị (Robust Normalization) sẽ chuyển đổi dữ liệu theo công thức:

$$X_{quan} = \frac{X - Q_2}{Q_3 - Q_1}$$

Trong đó, Q_1 , Q_2 và Q_3 là tứ phân vị của X .

```
[16]: # làm theo công thức
Q1, Q2, Q3 = df.before.quantile([.25, .5, .75])
df = df.assign(
    quan = (df.before - Q2) / (Q3 - Q1)
)
df.head()
```

```
[16]:   before    norm      st      bst    quan
0     5.1  0.222222 -0.897674 -0.900681 -0.538462
1     4.9  0.166667 -1.139200 -1.143017 -0.692308
2     4.7  0.111111 -1.380727 -1.385353 -0.846154
3     4.6  0.083333 -1.501490 -1.506521 -0.923077
4     5.0  0.194444 -1.018437 -1.021849 -0.615385
```

```
[17]: # sử dụng RobustScaler
rs = RobustScaler().fit(data)
rs.transform(data)[:5]
```

```
[17]: array([[ -0.53846154],
        [ -0.69230769],
        [ -0.84615385],
        [ -0.92307692],
        [ -0.61538462]])
```

```
[18]: #chuẩn bị dữ liệu
#chuẩn hóa theo min-max
odd_minmax = (odd - np.min(odd)) / (np.max(odd) - np.min(odd))
#chuẩn hóa theo điểm Z
odd_std = (odd - np.mean(odd)) / np.std(odd, ddof = 1)
#chuẩn hóa theo phân vị
q1, q2, q3 = np.quantile(odd, [0.25, 0.5, 0.75])
odd_quan = (odd - q2) / (q3 - q1)
```

```
[19]: #so sánh giữa các phương pháp chuẩn hóa
import matplotlib.pyplot as plt

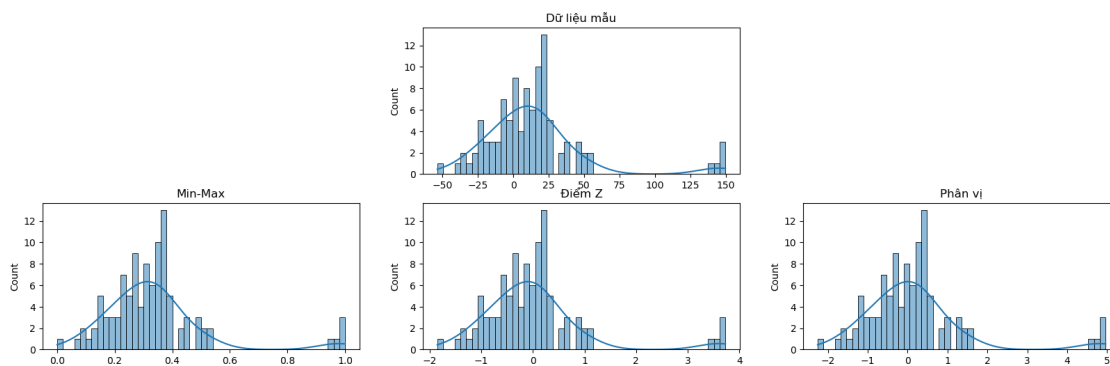
fig = plt.figure(figsize = (20, 6))
ax0 = fig.add_subplot(2, 3, 2)
```

```

ax1 = fig.add_subplot(2, 3, 4)
ax2 = fig.add_subplot(2, 3, 5)
ax3 = fig.add_subplot(2, 3, 6)

sns.histplot(odd, bins = 50, kde = True, ax = ax0)
ax0.title.set_text('Dữ liệu mẫu')
sns.histplot(odd_minmax, bins = 50, kde = True, ax = ax1)
ax1.title.set_text('Min-Max')
sns.histplot(odd_std, bins = 50, kde = True, ax = ax2)
ax2.title.set_text('Điểm Z')
sns.histplot(odd_quan, bins = 50, kde = True, ax = ax3)
ax3.title.set_text('Phân vị')

```



3 Một số điểm lưu ý khác

Khi chuẩn hóa dữ liệu, cần chú ý những điểm sau: - Chọn phương pháp chuẩn hóa phù hợp - Tránh rò rỉ dữ liệu

[]: